

Information Gathering Tool

Project Report

Name: Clement Boateng

Guided By: Prof. Dimple Chauhan

Table of Contents

Introduction.....	3
Project Requirements	3
Flow of the Tool.....	4
Flow Diagram	4
Design Decisions	5
Detail About the Script.....	5
Conclusion	12
References.....	13

Introduction

The information gathering tool, named **InfoGather**, is a command-line reconnaissance automation tool built in Python. It automates the collection of publicly available information about a specified domain or IP address, eliminating the need to run multiple reconnaissance tools manually.

During the information-gathering phase, security analysts run several different tools to understand their target. This tool combines some of these steps into a single program and also generates a report with the date and time it was created, which is automatically saved on the user's computer.

This document contains the requirements, design decisions, implementation details, and results of the finished tool.

Project Requirements

The project was scoped around the following core requirements:

- Accept a single target (a domain name or an IP address) as input from the user.
- Allow the user to choose which checks to run, rather than always running everything.
- Gather domain registration information.
- Gather DNS record information (A, MX, NS, TXT records).
- Check whether the target is reachable and measure its response time.
- Detect basic information about the web technology running on the target.
- Search the target's own publicly published web pages for email addresses.
- Combine all findings into a single, saved, human-readable report file.

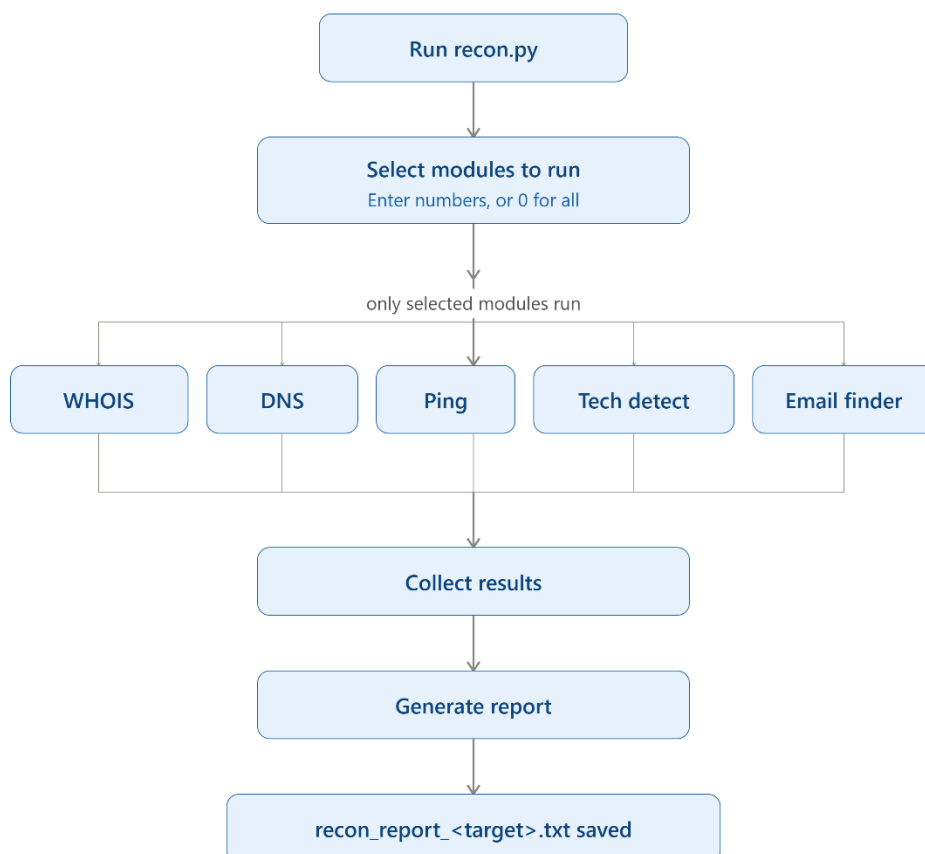
An additional requirement throughout the build was that the tool should not depend on any paid API keys or external paid services, keeping the tool free to run and simple to set up.

Flow of the Tool

InfoGather follows a simple, linear flow each time it is run:

1. The user runs the program (python3 recon.py).
2. The user enters the target which could be a domain name or IP address.
3. A menu is displayed listing all available modules the user can run.
4. The user types the number(s) of the module(s) they want, or 0 to run everything.
5. Only the selected modules are executed, one at a time, each returning its own result.
6. All the individual results are collected into a single list.
7. That list is passed to the report generator, which formats everything into readable text and saves it.

Flow Diagram



Design Decisions

A modular design was used, with each information-gathering technique placed in its own file. This reflects standard software engineering practice, and it directly shaped how the project could be built, tested, and extended. The following outlines the main benefits:

Isolated testing: Each module could be run and tested independently before integration. Any error can be fixed without needing to touch or risk breaking any other part of the tool.

Fault isolation: Modules handle their own errors, so a failure in one module or tool does not crash the whole program.

Extension: It is easy to include new functionalities without breaking what is already working.

Reusability: Since each module is self-contained, individual components can also be reused in other scripts or projects without making a single change.

Detail About the Script

This section provides a walkthrough of every file and explains how they work.

recon.py (The Main Program)

This is the main file the user runs. It loads all the modules, shows the menu, takes the user's input, and displays the results.

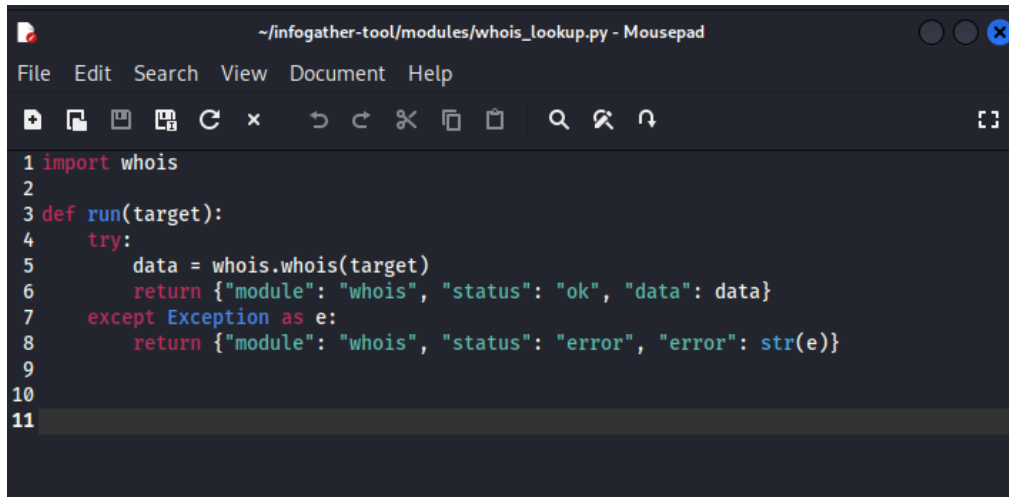
```

1 from modules import whois_lookup, dns_lookup, ping_check, tech_detect, email_harvest, report
2
3 MODULES = {
4     "1": ("Domain Registration Info", whois_lookup),
5     "2": ("Domain Server Records", dns_lookup),
6     "3": ("Website Connectivity Check", ping_check),
7     "4": ("Public Email Address Finder", email_harvest),
8     "5": ("Website Technology Detection", tech_detect),
9 }
10
11 def show_menu():
12     print("\nSelect modules to run:")
13     for key, (name, _) in MODULES.items():
14         print(f"[{key}] {name}")
15     print("[0] All")
16
17 def get_selection():
18     show_menu()
19     choice = input("Enter numbers (separated by comma) or 0 for all: ").strip()
20     if choice == "0":
21         return list(MODULES.keys())
22     return [c.strip() for c in choice.split(",") if c.strip() in MODULES]
23
24 def run_modules(target, selected_keys):
25     results = []
26     for key in selected_keys:
27         name, module = MODULES[key]
28         print(f"\nRunning {name}... ")
29         result = module.run(target)
30         results.append(result)
31         print(result)
32
33     print("\n— Summary —")
34     for r in results:
35         print(f"{r['module']}: {r['status']}")
36
37     report.generate(target, results)
38
39 def main():
40     target = input("Enter target (domain or IP): ").strip()
41
42     while True:
43         selected_keys = get_selection()
44
45         if not selected_keys:
46             print("No valid modules selected. Try again.")
47             continue
48
49         run_modules(target, selected_keys)
50
51         print("\nWhat would you like to do next?")
52         print("[m] Back to module menu (same target)")
53         print("[n] Enter a new target")
54         print("[q] Quit")
55         choice = input("Enter your choice: ").strip().lower()
56
57         if choice == "m":
58             continue
59         elif choice == "n":
60             target = input("Enter target (domain or IP): ").strip()
61             continue
62         else:
63             print("Goodbye.")
64             break
65
66 if __name__ == "__main__":
67     main()
68

```

whois_lookup.py (Domain Registration Lookup)

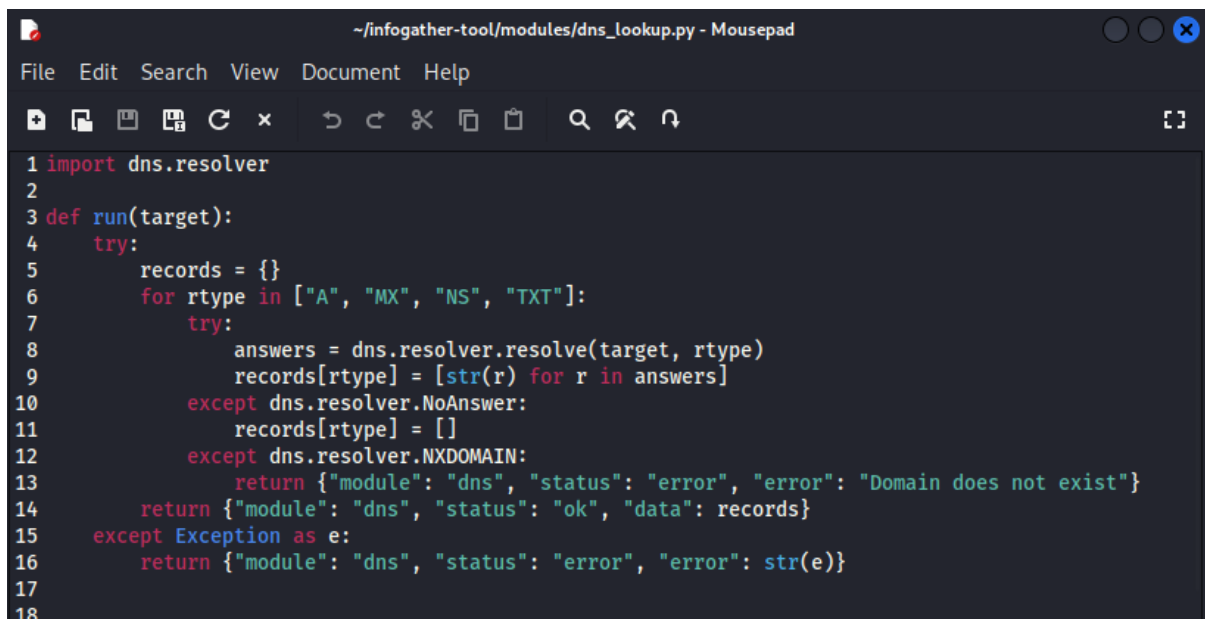
Uses the python-whois library to get information about a domain, such as the registrar, creation and expiration dates, name servers and if available, contact details.

A screenshot of a code editor window titled '~infogather-tool/modules/whois_lookup.py - Mousepad'. The editor has a menu bar with 'File', 'Edit', 'Search', 'View', 'Document', and 'Help'. Below the menu is a toolbar with icons for file operations and editing. The code is as follows:

```
1 import whois
2
3 def run(target):
4     try:
5         data = whois.whois(target)
6         return {"module": "whois", "status": "ok", "data": data}
7     except Exception as e:
8         return {"module": "whois", "status": "error", "error": str(e)}
9
10
11
```

dns_lookup.py (DNS Records Lookup)

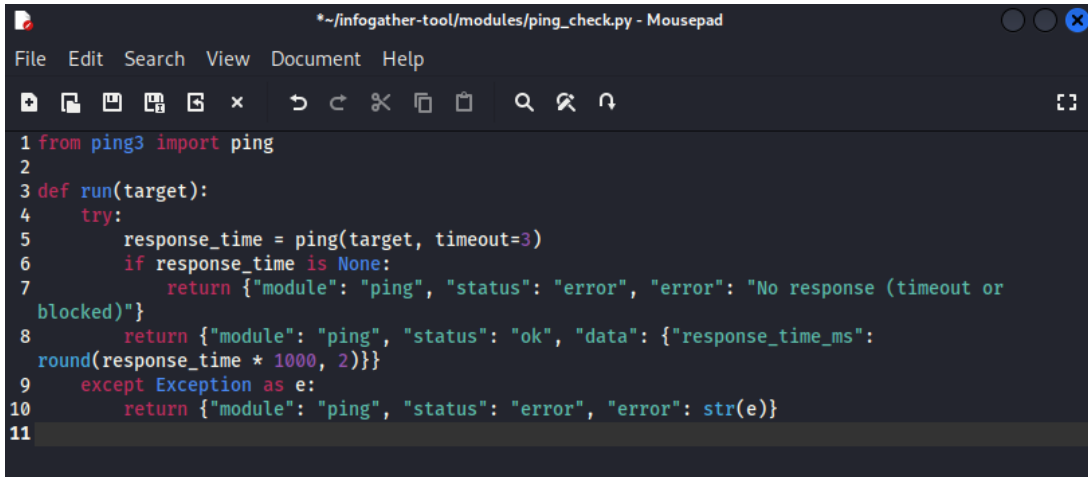
Uses the dnspython library to find four common DNS record types (A, MX, NS, and TXT) for the target. Each type is checked separately, so a missing record does not cause an error. Only a domain that does not exist (NXDOMAIN) is treated as an error.

A screenshot of a code editor window titled '~infogather-tool/modules/dns_lookup.py - Mousepad'. The editor has a menu bar with 'File', 'Edit', 'Search', 'View', 'Document', and 'Help'. Below the menu is a toolbar with icons for file operations and editing. The code is as follows:

```
1 import dns.resolver
2
3 def run(target):
4     try:
5         records = {}
6         for rtype in ["A", "MX", "NS", "TXT"]:
7             try:
8                 answers = dns.resolver.resolve(target, rtype)
9                 records[rtype] = [str(r) for r in answers]
10            except dns.resolver.NoAnswer:
11                records[rtype] = []
12            except dns.resolver.NXDOMAIN:
13                return {"module": "dns", "status": "error", "error": "Domain does not exist"}
14        return {"module": "dns", "status": "ok", "data": records}
15    except Exception as e:
16        return {"module": "dns", "status": "error", "error": str(e)}
17
18
```

ping_check.py (Connectivity Check)

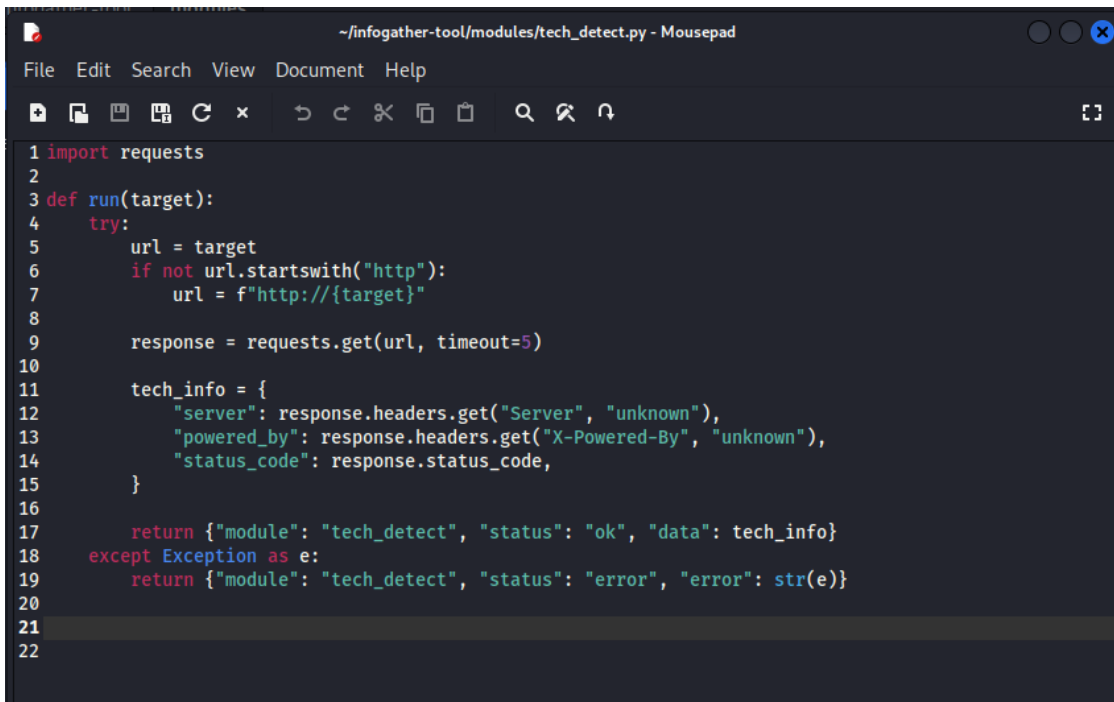
Uses the ping3 library to send a single ICMP ping to the target and measure the round-trip response time in milliseconds. This module requires elevated (root) privileges to send raw network packets, so it is run with sudo.



```
*~/infogather-tool/modules/ping_check.py - Mousepad
File Edit Search View Document Help
1 from ping3 import ping
2
3 def run(target):
4     try:
5         response_time = ping(target, timeout=3)
6         if response_time is None:
7             return {"module": "ping", "status": "error", "error": "No response (timeout or
            blocked)"}
8         return {"module": "ping", "status": "ok", "data": {"response_time_ms":
            round(response_time * 1000, 2)}}
9     except Exception as e:
10        return {"module": "ping", "status": "error", "error": str(e)}
11
```

tech_detect.py (Website Technology Detection)

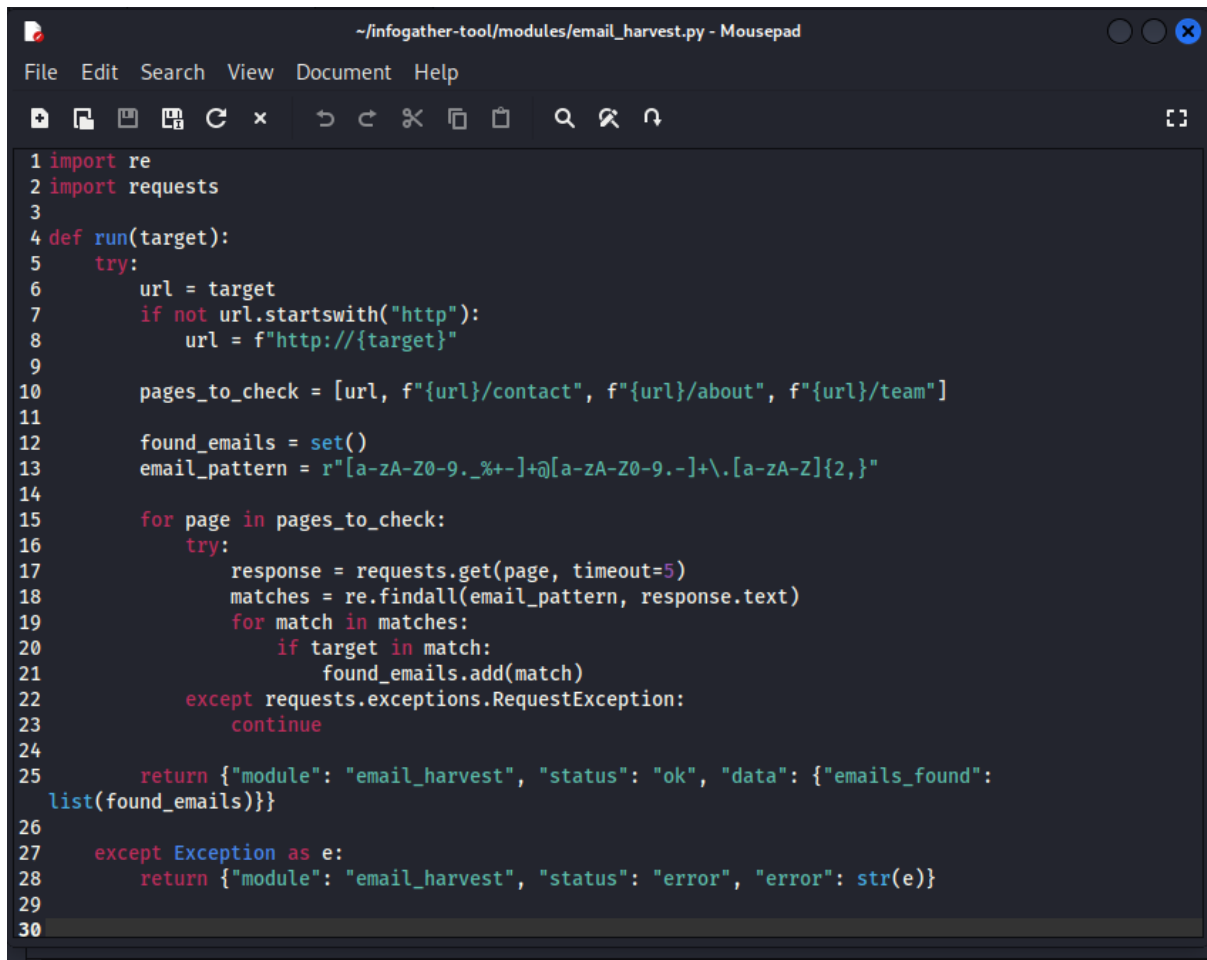
Sends a simple HTTP GET request to the target and checks the response headers, such as Server and X-Powered-By, to identify the web server software and the HTTP status code showing whether the site responded.



```
*~/infogather-tool/modules/tech_detect.py - Mousepad
File Edit Search View Document Help
1 import requests
2
3 def run(target):
4     try:
5         url = target
6         if not url.startswith("http"):
7             url = f"http://{target}"
8
9         response = requests.get(url, timeout=5)
10
11        tech_info = {
12            "server": response.headers.get("Server", "unknown"),
13            "powered_by": response.headers.get("X-Powered-By", "unknown"),
14            "status_code": response.status_code,
15        }
16
17        return {"module": "tech_detect", "status": "ok", "data": tech_info}
18    except Exception as e:
19        return {"module": "tech_detect", "status": "error", "error": str(e)}
20
21
22
```


email_harvest.py (Public Email Address Finder)

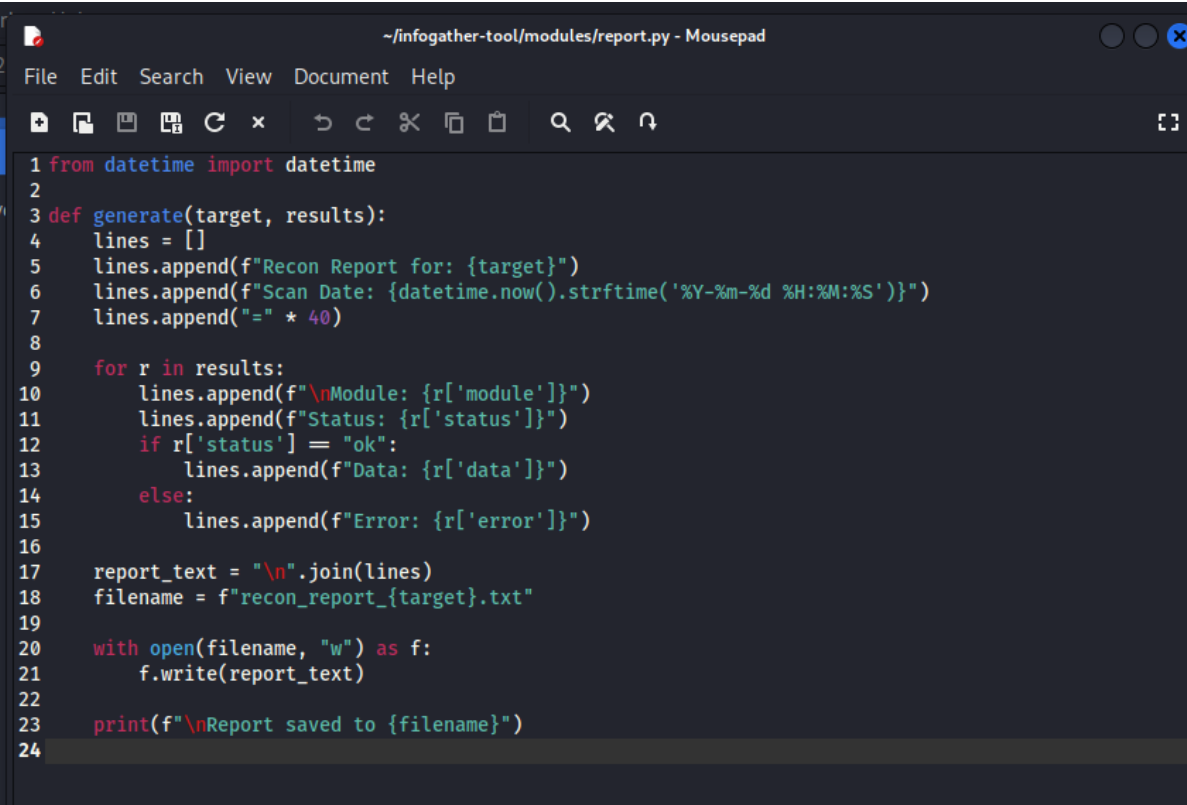
Checks the target's homepage and common pages like /contact, /about, and /team for publicly visible email addresses. It only keeps addresses from the target's own domain and removes duplicates.

A screenshot of a code editor window titled "~infogather-tool/modules/email_harvest.py - Mousepad". The editor has a menu bar with "File", "Edit", "Search", "View", "Document", and "Help". Below the menu is a toolbar with icons for file operations and editing. The code is written in Python and is as follows:

```
1 import re
2 import requests
3
4 def run(target):
5     try:
6         url = target
7         if not url.startswith("http"):
8             url = f"http://{target}"
9
10        pages_to_check = [url, f"{url}/contact", f"{url}/about", f"{url}/team"]
11
12        found_emails = set()
13        email_pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
14
15        for page in pages_to_check:
16            try:
17                response = requests.get(page, timeout=5)
18                matches = re.findall(email_pattern, response.text)
19                for match in matches:
20                    if target in match:
21                        found_emails.add(match)
22            except requests.exceptions.RequestException:
23                continue
24
25        return {"module": "email_harvest", "status": "ok", "data": {"emails_found":
list(found_emails)}}
26
27    except Exception as e:
28        return {"module": "email_harvest", "status": "error", "error": str(e)}
29
30
```

report.py (Report Generation)

Receives the target and all the results collected from the modules. It creates a report that includes the title, the date and time and the results from each module. If a module has an error, the error is also included in the report. The report is saved as a text file named recon_report_<target>.txt.



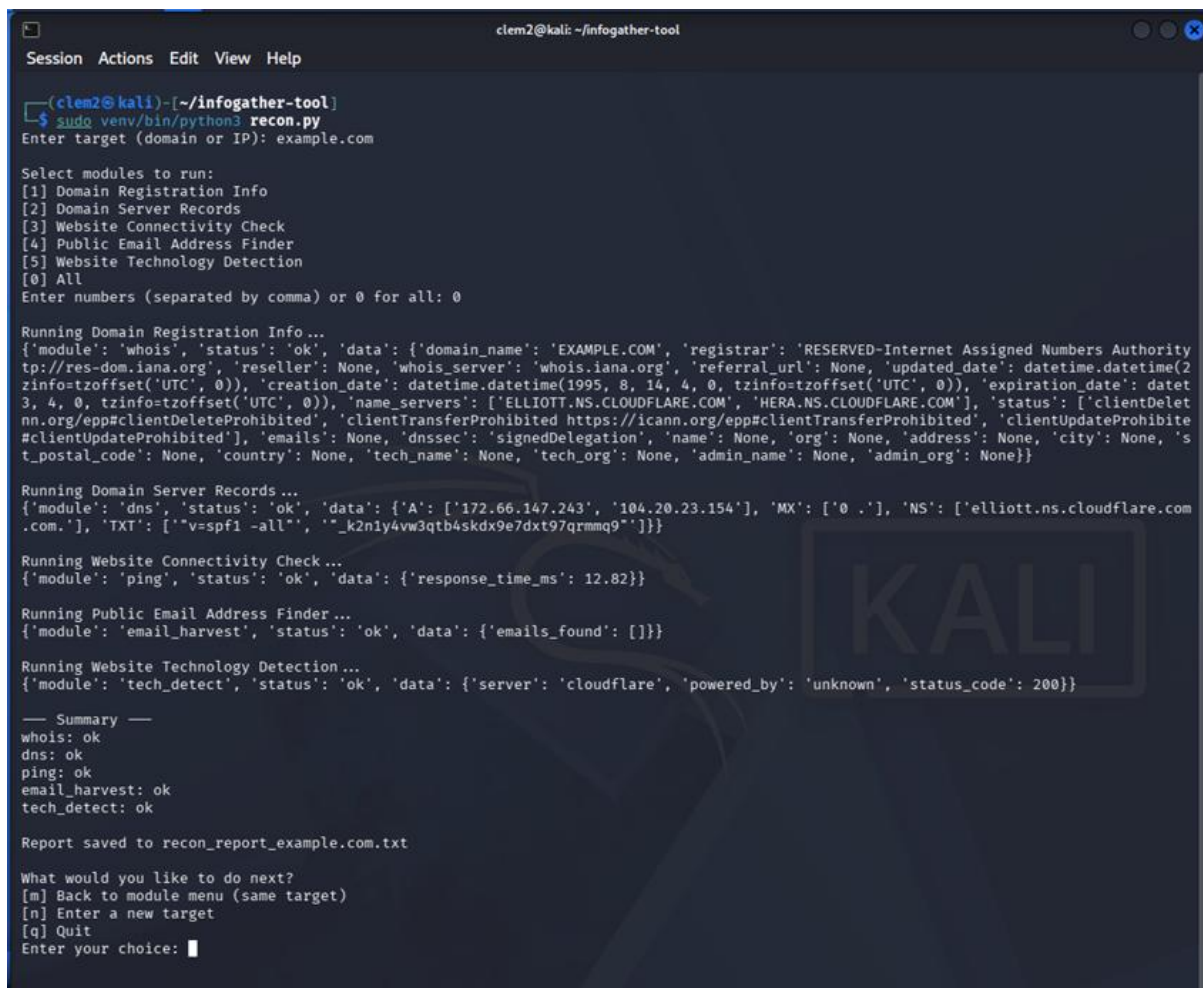
The screenshot shows a text editor window titled `~/infogather-tool/modules/report.py - Mousepad`. The menu bar includes `File`, `Edit`, `Search`, `View`, `Document`, and `Help`. The toolbar contains icons for file operations (new, open, save, print, close), editing (undo, redo, cut, copy, paste), and search (find, replace, go to line). The code is as follows:

```

1 from datetime import datetime
2
3 def generate(target, results):
4     lines = []
5     lines.append(f"Recon Report for: {target}")
6     lines.append(f"Scan Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
7     lines.append("=" * 40)
8
9     for r in results:
10        lines.append(f"\nModule: {r['module']}")
11        lines.append(f"Status: {r['status']}")
12        if r['status'] == "ok":
13            lines.append(f>Data: {r['data']}")
14        else:
15            lines.append(f>Error: {r['error']}")
16
17    report_text = "\n".join(lines)
18    filename = f"recon_report_{target}.txt"
19
20    with open(filename, "w") as f:
21        f.write(report_text)
22
23    print(f"\nReport saved to {filename}")
24

```

Full Run (All Modules Selected)



```
clem2@kali: ~/infogather-tool
Session Actions Edit View Help

(clem2@kali)~/infogather-tool
$ sudo venv/bin/python3 recon.py
Enter target (domain or IP): example.com

Select modules to run:
[1] Domain Registration Info
[2] Domain Server Records
[3] Website Connectivity Check
[4] Public Email Address Finder
[5] Website Technology Detection
[0] All
Enter numbers (separated by comma) or 0 for all: 0

Running Domain Registration Info...
{'module': 'whois', 'status': 'ok', 'data': {'domain_name': 'EXAMPLE.COM', 'registrar': 'RESERVED-Internet Assigned Numbers Authority
tp://res-dom.iana.org', 'reseller': None, 'whois_server': 'whois.iana.org', 'referral_url': None, 'updated_date': datetime.datetime(2
zinfo=tzoffset('UTC', 0)), 'creation_date': datetime.datetime(1995, 8, 14, 4, 0, tzinfo=tzoffset('UTC', 0)), 'expiration_date': datet
3, 4, 0, tzinfo=tzoffset('UTC', 0)), 'name_servers': ['ELLIOTT.NS.CLOUDFLARE.COM', 'HERA.NS.CLOUDFLARE.COM'], 'status': ['clientDelet
nn.org/epp#clientDeleteProhibited', 'clientTransferProhibited https://icann.org/epp#clientTransferProhibited', 'clientUpdateProhibite
#clientUpdateProhibited'], 'emails': None, 'dnssec': 'signedDelegation', 'name': None, 'org': None, 'address': None, 'city': None, 's
t_postal_code': None, 'country': None, 'tech_name': None, 'tech_org': None, 'admin_name': None, 'admin_org': None}}

Running Domain Server Records...
{'module': 'dns', 'status': 'ok', 'data': {'A': ['172.66.147.243', '104.20.23.154'], 'MX': ['0 .'], 'NS': ['elliott.ns.cloudflare.com
.com.'], 'TXT': ['v=spf1 -all', '_k2n1y4vw3qtb4skdx9e7dxt97qrmmq9']}}

Running Website Connectivity Check...
{'module': 'ping', 'status': 'ok', 'data': {'response_time_ms': 12.82}}

Running Public Email Address Finder...
{'module': 'email_harvest', 'status': 'ok', 'data': {'emails_found': []}}

Running Website Technology Detection...
{'module': 'tech_detect', 'status': 'ok', 'data': {'server': 'cloudflare', 'powered_by': 'unknown', 'status_code': 200}}

— Summary —
whois: ok
dns: ok
ping: ok
email_harvest: ok
tech_detect: ok

Report saved to recon_report_example.com.txt

What would you like to do next?
[m] Back to module menu (same target)
[n] Enter a new target
[q] Quit
Enter your choice: █
```

Selective Run (Chosen Modules Only)

The screenshot below demonstrates the tool's selective execution feature, showing that only the selected modules 1, 3, and 5 were run.

```
clem2@kali: ~/infogather-tool
Session Actions Edit View Help

(clem2@kali)~/infogather-tool
$ sudo venv/bin/python3 recon.py
Enter target (domain or IP): example.com

Select modules to run:
[1] Domain Registration Info
[2] Domain Server Records
[3] Website Connectivity Check
[4] Public Email Address Finder
[5] Website Technology Detection
[0] All
Enter numbers (separated by comma) or 0 for all: 1,3,5

Running Domain Registration Info...
{'module': 'whois', 'status': 'ok', 'data': {'domain_name': 'EXAMPLE.COM', 'registrar': 'RESERVED-Internet Assigned Numbers Authority', 'registrar_url': 'http://res-dom.iana.org', 'reseller': None, 'whois_server': 'whois.iana.org', 'referral_url': None, 'updated_date': datetime.datetime(2026, 8, 14, 8, 1, 43, tzinfo=tzoffset('UTC', 0)), 'creation_date': datetime.datetime(1995, 8, 14, 4, 0, tzinfo=tzoffset('UTC', 0)), 'expiration_date': datetime.datetime(2027, 8, 13, 4, 0, tzinfo=tzoffset('UTC', 0)), 'name_servers': ['ELLIOTT.NS.CLOUDFLARE.COM', 'HERA.NS.CLOUDFLARE.COM'], 'status': ['clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited', 'clientTransferProhibited https://icann.org/epp#clientTransferProhibited', 'clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited'], 'emails': None, 'dnssec': 'signedDelegation', 'name': None, 'org': None, 'address': None, 'city': None, 'state': None, 'registrant_postal_code': None, 'country': None, 'tech_name': None, 'tech_org': None, 'admin_name': None, 'admin_org': None}}

Running Website Connectivity Check...
{'module': 'ping', 'status': 'ok', 'data': {'response_time_ms': 9.77}}

Running Website Technology Detection...
{'module': 'tech_detect', 'status': 'ok', 'data': {'server': 'cloudflare', 'powered_by': 'unknown', 'status_code': 200}}

— Summary —
whois: ok
ping: ok
tech_detect: ok

Report saved to recon_report_example.com.txt

What would you like to do next?
[m] Back to module menu (same target)
[n] Enter a new target
[q] Quit
Enter your choice: █
```

Conclusion

InfoGather successfully meets all of the project's original requirements. It combines five reconnaissance techniques, WHOIS lookup, DNS record lookup, connectivity checking, website technology detection, and public email address discovery into one simple command-line tool. Users can choose to run all checks or only the modules they need, and each run produces a clear, saved report.

Using separate modules made the tool easier to test, debug, and update. Each module can handle its own errors, allowing the rest of the program to continue running if one module fails.

The final version brings all the required features together into a single working tool, providing a simple way to gather information and save the results for the user.

References

1. python-whois (PyPI) (Python library for querying WHOIS servers.)
2. dnspython (PyPI) (Python library for performing DNS queries.)
3. ping3 (PyPI) (Python library for sending ICMP ping requests.)
4. Requests (PyPI) (Python HTTP library used for web requests in the technology detection and email harvesting modules.)
5. example.com (Used for testing)
6. Python Software Foundation (Official Python 3 documentation, including the re, datetime, and argparse standard library modules)